



Karnataka Digital
Economy Mission



K-VLSI

VERILOG PROGRAMS

NAME: DHANAPAL K

KVLSI ID: KVLSI2501061

Under the guidance of :

Shashi Kant Sharma

INDEX

Sl No.	CONTENTS	Page No.
1.	Verilog Datatypes (default size and default value)	01 – 03
2.	Verilog Operators	04 – 07
3.	Verilog Format Specifiers and System Task	08 – 10
4.	Blocking and Non-Blocking Assignments	11 – 12
5.	Race Condition in Blocking and Non-Blocking Assignment	13 – 15
6.	Fibonacci Series	16
7.	Prime number checker	17
8.	Even or Odd number checker	18
9.	To find largest and smallest number	19
10.	Sorting Array in Ascending Order (Bubble Sort)	20
11.	Left-aligned triangle star pattern	21
12.	Right-aligned triangle star pattern	22
13.	Inverted left triangle star pattern	23
14.	Inverted right triangle star pattern	24
15.	Pyramid star pattern	25
16.	Inverted pyramid star pattern	26
17.	Diamond star pattern	27
18.	Left aligned triangle pattern	28
19.	Right aligned triangle pattern	29
20.	Inverted left aligned triangle pattern	30
21.	Inverted right aligned triangle pattern	31
22.	Adder using generate-for	32 – 33
23.	Adder using generate-if (Select between 2-bit and 4-bit Adder)	34 – 35
24.	Half Adder using functions	36 – 37
25.	Full Adder using functions	38 – 39
26.	Ripple Carry Adder using function (4 bits)	40 – 41
27.	Function to check whether the number is prime or not	42 – 43
28.	Function to check whether the number is even or not	44
29.	Fibonacci Series using function	45 – 46
30.	Half Adder using tasks	47 – 48
31.	Full Adder using tasks	49 – 50
32.	Clock generation using task	51 – 52
33.	Multiple ways to generate a clock	53 – 56
34.	Clock generation of 70% duty cycle	57
35.	Clock generation of 40% duty cycle	58
36.	Clock generation for 5ns time period	59
37.	Clock generation for 6.6ns time period	60
38.	Clock generation for 4.4microsecond time period	61
39.	Verilog code to generate 10 unique random numbers between 0-500	62 – 63

40.	Verilog code to generate 20 unique random numbers between 0-1000 in ascending order	64 – 66
-----	---	---------

1. Verilog Datatypes (default size and default value)

- **Code:**

```
module verilog_datatypes_tb;

    // wire — used for continuous assignment (default: z / unknown if not driven)
    wire w1, w2;
    wire [3:0] w_bus;

    // reg — used in procedural blocks (default: x / unknown)
    reg r1, r2;
    reg [7:0] r_bus;

    // integer — 32-bit signed (default: x)
    integer count;

    // real — double-precision floating-point (default: undefined)
    real pi, area;

    // time — 64-bit unsigned integer (default: 0)
    time start_time, end_time;

    // parameter — constant, no default, must be initialized
    parameter WIDTH = 8;

    // localparam — like parameter but not overrideable
    localparam DEPTH = 16;

    // string (Verilog workaround): array of characters (default: all x)
    reg [8*10:1] string_example;

    // Assign values to wire
    assign w1 = 1'b1;
    assign w2 = 1'b0;
    assign w_bus = 4'b1010;

    initial
    begin
        $display("=== Verilog Data Types: Default Values and Sizes ===");

        // Display default values before assignment
        $display("\n--- Before Initialization ---");
        $display("reg r1    = %b (Size: %0d bits)", r1, $bits(r1));
        $display("reg r2    = %b (Size: %0d bits)", r2, $bits(r2));
        $display("reg r_bus = %b (Size: %0d bits)", r_bus, $bits(r_bus));
        $display("integer  = %0d (Size: %0d bits)", count, $bits(count));
        $display("real pi   = %f", pi);
        $display("time start = %0t (Size: %0d bits)", start_time, $bits(start_time));
        $display("string_example = %s", string_example);
    end
endmodule
```

```

// Initialize all
r1 = 1;
r2 = 0;
r_bus = 8'hFF;

count = -25;
pi = 3.14159;
area = pi * 2.0 * 2.0;

start_time = $time;
#10;
end_time = $time;

string_example = "Dhanapal K";

// Display after initialization
$display("\n--- After Initialization ---");
$display("wire w1    = %b (Size: %0d bits)", w1, $bits(w1));
$display("wire w2    = %b", w2);
$display("wire w_bus = %b (Size: %0d bits)", w_bus, $bits(w_bus));

$display("reg r1     = %b", r1);
$display("reg r2     = %b", r2);
$display("reg r_bus  = %b", r_bus);

$display("integer   = %0d", count);
$display("real area  = %0f", area);
$display("time elapsed= %0t ns", end_time - start_time);

$display("parameter WIDTH = %0d", WIDTH);
$display("localparam DEPTH = %0d", DEPTH);

$display("string_example = %s (Size: %0d bits)", string_example, $bits(string_example));

$finish;
end
endmodule

```

- **OUTPUT:**

=== Verilog Data Types: Default Values and Sizes ===

```

--- Before Initialization ---
reg r1    = x (Size: 1 bits)
reg r2    = x (Size: 1 bits)
reg r_bus = xxxxxxxx (Size: 8 bits)
integer   = x (Size: 32 bits)
real pi   = 0.000000
time start = 0 (Size: 64 bits)

```

string_example =

--- After Initialization ---

wire w1 = 1 (Size: 1 bits)

wire w2 = 0

wire w_bus = 1010 (Size: 4 bits)

reg r1 = 1

reg r2 = 0

reg r_bus = 11111111

integer = -25

real area = 12.566360

time elapsed= 10000 ns

parameter WIDTH = 8

localparam DEPTH = 16

string_example = Dhanapal K (Size: 80 bits)

- **EDA Playground Link:**

<https://edaplayground.com/x/9kU6>

2. Verilog Operators

- **Code:**

```
module operators_demo_tb;

    // Operands
    reg [3:0] a, b;
    reg [7:0] x;
    reg signed [3:0] sa, sb;
    reg [7:0] data;
    reg flag;

    initial
    begin
        // Initialize operands
        a = 4'b1010; // 10
        b = 4'b0101; // 5
        sa = -4;
        sb = 2;
        data = 8'b11001100;
        flag = 1'b1;

        $display("=== Arithmetic Operators ===");
        $display("a + b = %0d", a + b); // 15
        $display("a - b = %0d", a - b); // 5
        $display("a * b = %0d", a * b); // 50
        $display("a / b = %0d", a / b); // 2
        $display("a %% b = %0d", a % b); // 0
        $display("-sa = %0d", -sa); // 4

        $display("\n=== Relational Operators ===");
        $display("a == b : %b", a == b);
        $display("a != b : %b", a != b);
        $display("a > b : %b", a > b);
```

$\$display("a < b : \%b", a < b);$

$\$display("a >= b : \%b", a >= b);$

$\$display("a <= b : \%b", a <= b);$

$\$display("\n=== Logical Operators ===");$

$\$display("a \&\& b = \%b", a \&\& b);$

$\$display("a \parallel b = \%b", a \parallel b);$

$\$display("!a = \%b", !a);$

$\$display("\n=== Bitwise Operators ===");$

$\$display("a \& b = \%b", a \& b);$

$\$display("a | b = \%b", a | b);$

$\$display("a \wedge b = \%b", a \wedge b);$

$\$display("\sim a = \%b", \sim a);$

$\$display("\n=== Shift Operators ===");$

$\$display("a << 1 = \%b", a << 1);$

$\$display("a >> 1 = \%b", a >> 1);$

$\$display("sa <<< 1 = \%b", sa <<< 1);$

$\$display("sa >>> 1 = \%b", sa >>> 1);$

$\$display("\n=== Reduction Operators ===");$

$\$display("&a = \%b", \&a);$ // AND all bits

$\$display("|a = \%b", |a);$ // OR all bits

$\$display("^a = \%b", ^a);$ // XOR all bits

$\$display("\sim^a = \%b", \sim^a);$ // XNOR reduction

$\$display("\n=== Conditional (Ternary) Operator ===");$

$\$display("flag ? a : b = \%0d", flag ? a : b);$

$\$display("\n=== Concatenation and Replication ===");$


```
$display("{a,b}    = %b", {a,b});
```

```
$display("{2{a}}    = %b", {2{a}});
```

```
$display("\n=== Assignment Operators ===");
```

```
x = a + b; $display("x = a + b => %0d", x);
```

```
x += 1;  $display("x += 1    => %0d", x);
```

```
x -= 2;  $display("x -= 2    => %0d", x);
```

```
x *= 2;  $display("x *= 2    => %0d", x);
```

```
x /= 3;  $display("x /= 3    => %0d", x);
```

```
$display("\n=== Bit Select, Part Select ===");
```

```
$display("data[3]    = %b", data[3]);
```

```
$display("data[7:4]  = %b", data[7:4]);
```

```
$finish;
```

```
end
```

```
endmodule
```

• OUTPUT:

```
=== Arithmetic Operators ===
```

```
a + b = 15
```

```
a - b = 5
```

```
a * b = 2
```

```
a / b = 2
```

```
a % b = 0
```

```
-sa  = 4
```

```
=== Relational Operators ===
```

```
a == b : 0
```

```
a != b : 1
```

```
a > b : 1
```

```
a < b : 0
```

```
a >= b : 1
```

```
a <= b : 0
```

```
=== Logical Operators ===
```

```
a && b = 1
```

```
a || b = 1
```

!a = 0

==== Bitwise Operators ====

a & b = 0000

a | b = 1111

a ^ b = 1111

~a = 0101

==== Shift Operators ====

a << 1 = 0100

a >> 1 = 0101

sa <<< 1 = 1000

sa >>> 1 = 1110

==== Reduction Operators ====

&a = 0

|a = 1

^a = 0

~^a = 1

==== Conditional (Ternary) Operator ====

flag ? a : b = 10

==== Concatenation and Replication ====

{a,b} = 10100101

{2{a}} = 10101010

==== Assignment Operators ====

x = a + b => 15

x += 1 => 16

x -= 2 => 14

x *= 2 => 28

x /= 3 => 9

==== Bit Select, Part Select ====

data[3] = 1

data[7:4] = 1100

- **EDA Playground Link:**

<https://edaplayground.com/x/HyJk>

3. Verilog Format Specifiers and System Task

- **Code:**

```
module format_and_system_tasks_demo;

// Test variables
reg [3:0] nibble = 4'b1010;
reg [7:0] byte_data = 8'hAF;
integer i = -42;
real r = 3.14159;
time t;
reg [7:0] char_val = 8'd65; // ASCII 'A'

initial
begin
// Time task
t = $time;

$display("Simulation Time: %0t ns", t);

$display("\n==== Format Specifiers =====");

// Binary, Octal, Decimal, Hex
$display("%%b (Binary)      : %b", nibble);    // binary
$display("%%o (Octal)      : %o", nibble);    // octal
$display("%%d (Decimal)     : %d", nibble);    // decimal (signed)
$display("%%h (Hex)         : %h", nibble);    // hex
$display("%%x (Hex)         : %x", nibble);    // hex lowercase

// Signed and Unsigned
$display("%%d (Signed int)   : %d", i);
$display("%%u (Unsigned int) : %u", i);
```

// Character and String

```
$display("%%c (Char from int) : %c", char_val); // outputs 'A'
```

```
$display("%%s (String) : %s", "Hello");
```

// Real number

```
$display("%%f (Real float) : %f", r);
```

```
$display("%%e (Scientific) : %e", r);
```

```
$display("%%g (Compact float) : %g", r);
```

// Display with width

```
$display("%%0d (Zero pad 4b) : %04b", nibble);
```

```
$display("Field Width %%4d : %4d", nibble);
```

```
$display("\n==== System Tasks ====");
```

// \$display (adds newline)

```
$display("This is $display: prints with newline");
```

// \$write (no newline)

```
$write("This is $write: ");
```

```
$write("prints inline without newline\n");
```

// \$strobe: Executes after all non-blocking assignments

```
#0 $strobe("This is $strobe: after NBA");
```

// \$monitor: Automatically displays on value change

```
$monitor("Monitor -> Time=%0t, i=%0d, r=%0f", $time, i, r);
```

```
i = 10; #1;
```

```
i = 20; #1;
```

```
r = 6.28; #1;
```

```
$finish;  
end  
endmodule
```

- **OUTPUT:**

==== Format Specifiers =====

```
%b (Binary)      : 1010  
%o (Octal)       : 12  
%d (Decimal)     : 10  
%h (Hex)         : a  
%x (Hex)         : a  
%d (Signed int)  : -42  
%u (Unsigned int): ????  
%c (Char from int): A  
%s (String)      : Hello  
%f (Real float)  : 3.141590  
%e (Scientific)  : 3.141590e+00  
%g (Compact float): 3.14159  
%0d (Zero pad 4b): 1010  
Field Width %4d  : 10
```

==== System Tasks =====

```
This is $display: prints with newline  
This is $write: prints inline without newline  
Monitor -> Time=0, i=10, r=3.141590  
This is $strobe: after NBA  
Monitor -> Time=1000, i=20, r=3.141590  
Monitor -> Time=2000, i=20, r=6.280000
```

- **EDA Playground Link:**

<https://edaplayground.com/x/gh6Q>

4. Blocking and Non-Blocking Assignments

- **Code:**

```
module blocking_vs_nonblocking;
    reg a, b, c;
    reg x, y, z;
    initial
    begin
        $display("=== Blocking Assignment ===");
        a = 1'b0;
        b = 1'b0;
        c = 1'b0;

        // Blocking assignment: executes sequentially
        a = 1;
        b = a;
        c = b;

        $display("After blocking assignments:");
        $display("a = %b, b = %b, c = %b", a, b, c);
    end

    initial
    begin
        #10; // Wait a bit to distinguish outputs
        $display("\n=== Non-Blocking Assignment ===");
        x = 1'b0;
        y = 1'b0;
        z = 1'b0;

        // Non-blocking assignment: all right-hand sides are evaluated first
```

```
x <= 1;  
y <= x;  
z <= y;  
#1; // Wait 1 time unit to see result  
$display("After non-blocking assignments:");  
$display("x = %b, y = %b, z = %b", x, y, z);  
end  
endmodule
```

- **OUTPUT:**

=== Blocking Assignment ===

After blocking assignments:

a = 1, b = 1, c = 1

=== Non-Blocking Assignment ===

After non-blocking assignments:

x = 1, y = 0, z = 0

- **EDA Playground Link:**

<https://edaplayground.com/x/vEAA>

5. Race Condition in Blocking and Non-Blocking Assignment

- **Code:**

```
module race_condition_demo;

// Case 1: a=10, b=20
reg [7:0] a_block1, b_block1;
reg [7:0] a_nblk1, b_nblk1;

// Case 2: a=20, b=10
reg [7:0] a_block2, b_block2;
reg [7:0] a_nblk2, b_nblk2;

// === CASE 1 ===
initial
begin
    a_block1 = 10; b_block1 = 20;
    a_nblk1 = 10; b_nblk1 = 20;

    $display("\n=== CASE 1: a=10, b=20 ===");
    $display("Before Assignment:");
    $display("[BLOCKING]   a = %0d, b = %0d", a_block1, b_block1);
    $display("[NON-BLOCKING] a = %0d, b = %0d", a_nblk1, b_nblk1);
end

// Blocking: Race-prone
always @(*) a_block1 = b_block1;
always @(*) b_block1 = a_block1;

// Non-Blocking: Race-free
always @(*) a_nblk1 <= b_nblk1;
```



```
always @(*) b_nblk1 <= a_nblk1;

// === CASE 2 ===

initial
begin
    #5;
    a_block2 = 20; b_block2 = 10;
    a_nblk2 = 20; b_nblk2 = 10;
    $display("\n=== CASE 2: a=20, b=10 ===");
    $display("Before Assignment:");
    $display("[BLOCKING]   a = %0d, b = %0d", a_block2, b_block2);
    $display("[NON-BLOCKING] a = %0d, b = %0d", a_nblk2, b_nblk2);
end

// Blocking: Race-prone
always @(*)
    a_block2 = b_block2;
always @(*)
    b_block2 = a_block2;

// Non-Blocking: Race-free
always @(*)
    a_nblk2 <= b_nblk2;
always @(*)
    b_nblk2 <= a_nblk2;

// Display Final Results
initial
begin
    #10;
    $display("\nAfter Assignment:");
    $display("=== CASE 1 ===");
```

```

$display("[BLOCKING]   a = %0d, b = %0d", a_block1, b_block1);
$display("[NON-BLOCKING] a = %0d, b = %0d", a_nblk1, b_nblk1);
$display("\n=== CASE 2 ===");
$display("[BLOCKING]   a = %0d, b = %0d", a_block2, b_block2);
$display("[NON-BLOCKING] a = %0d, b = %0d", a_nblk2, b_nblk2);

$finish;

end

endmodule

```

- **OUTPUT:**

=== CASE 1: a=10, b=20 ===

Before Assignment:

[BLOCKING] a = 10, b = 20

[NON-BLOCKING] a = 10, b = 20

=== CASE 2: a=20, b=10 ===

Before Assignment:

[BLOCKING] a = 20, b = 10

[NON-BLOCKING] a = 20, b = 10

After Assignment:

=== CASE 1 ===

[BLOCKING] a = 10, b = 10 // May vary (race)

[NON-BLOCKING] a = 20, b = 10 // Always correct

=== CASE 2 ===

[BLOCKING] a = 20, b = 20 // May vary (race)

[NON-BLOCKING] a = 10, b = 20 // Always correct

- **EDA Playground Link:**

<https://edaplayground.com/x/ZFP5>

6. Fibonacci Series

DESIGN CODE	TESTBENCH CODE
<pre> module fibonacci #(parameter N = 10); integer fib [0:N-1]; integer i; initial begin fib[0] = 0; fib[1] = 1; for (i = 2; i < N; i = i + 1) fib[i] = fib[i-1] + fib[i-2]; end endmodule </pre>	<pre> module fibonacci_tb; parameter N = 10; fibonacci #(N) uut(); initial begin #1; \$display("Fibonacci Series:"); for (integer i = 0; i < N; i = i + 1) \$display("fib[%0d] = %0d", i, uut.fib[i]); \$finish; end endmodule </pre>
OUTPUT	
<pre> Fibonacci Series: fib[0] = 0 fib[1] = 1 fib[2] = 1 fib[3] = 2 fib[4] = 3 fib[5] = 5 fib[6] = 8 fib[7] = 13 fib[8] = 21 fib[9] = 34 </pre>	

- **EDA Playground Link:**
<https://edaplayground.com/x/preT>

7. Prime number checker

DESIGN CODE	TESTBENCH CODE
<pre> module prime_check(input [7:0] num, output reg is_prime); integer i; always @(*) begin if (num <= 1) is_prime = 0; else begin is_prime = 1; for (i = 2; i <= num / 2; i = i + 1) if (num % i == 0) begin is_prime = 0; break; end end end end endmodule </pre>	<pre> module prime_check_tb; reg [7:0] num; wire is_prime; prime_check uut(num, is_prime); initial begin num = 29; #1; \$display("%0d is %sprime", num, is_prime ? "" : "not "); num = 30; #1; \$display("%0d is %sprime", num, is_prime ? "" : "not "); \$finish; end endmodule </pre>
OUTPUT	
<pre> 29 is prime 30 is not prime </pre>	

- **EDA Playground Link:**
<https://edaplayground.com/x/sbcq>

8. Even or Odd number checker

DESIGN CODE	TESTBENCH CODE
<pre> module even_odd(input [7:0] num, output reg is_even); always @(num) begin is_even = (num % 2 == 0); end endmodule </pre>	<pre> module even_odd_tb; reg [7:0] num; wire is_even; even_odd uut(num, is_even); initial begin num = 45; #1; \$display("%0d is %s", num, is_even ? "Even" : "Odd"); num = 54; #1; \$display("%0d is %s", num, is_even ? "Even" : "Odd"); \$finish; end endmodule </pre>
OUTPUT	
<pre> 45 is Odd 54 is Even </pre>	

- **EDA Playground Link:**
<https://edaplayground.com/x/puiiu>

9. To find largest and smallest number

• Design Code	• Testbench Code																														
<pre>module find_min_max (A, B, C, max_val, min_val); input [7:0] A, B, C; output reg [7:0] max_val, min_val; always @(A,B,C) begin // Find maximum if (A >= B && A >= C) max_val = A; else if (B >= A && B >= C) max_val = B; else max_val = C; // Find minimum if (A <= B && A <= C) min_val = A; else if (B <= A && B <= C) min_val = B; else min_val = C; end endmodule</pre>	<pre>module find_min_max_tb; reg [7:0] A, B, C; wire [7:0] max_val, min_val; find_min_max uut (A, B, C, max_val, min_val); initial begin \$display("Time\tA\tB\tC\tMax\tMin"); A = 8'd25; B = 8'd90; C = 8'd45; #10; \$display("%0t\t%d\t%d\t%d\t%d\t%d", \$time, A, B, C, max_val, min_val); A = 8'd5; B = 8'd5; C = 8'd5; #10; \$display("%0t\t%d\t%d\t%d\t%d\t%d", \$time, A, B, C, max_val, min_val); A = 8'd100; B = 8'd20; C = 8'd99; #10; \$display("%0t\t%d\t%d\t%d\t%d\t%d", \$time, A, B, C, max_val, min_val); A = 8'd3; B = 8'd8; C = 8'd1; #10; \$display("%0t\t%d\t%d\t%d\t%d\t%d", \$time, A, B, C, max_val, min_val); \$finish; end endmodule</pre>																														
• Output																															
<table><tr><th>Time</th><th>A</th><th>B</th><th>C</th><th>Max</th><th>Min</th></tr><tr><td>10</td><td>25</td><td>90</td><td>45</td><td>90</td><td>25</td></tr><tr><td>20</td><td>5</td><td>5</td><td>5</td><td>5</td><td>5</td></tr><tr><td>30</td><td>100</td><td>20</td><td>99</td><td>100</td><td>20</td></tr><tr><td>40</td><td>3</td><td>8</td><td>1</td><td>8</td><td>1</td></tr></table>		Time	A	B	C	Max	Min	10	25	90	45	90	25	20	5	5	5	5	5	30	100	20	99	100	20	40	3	8	1	8	1
Time	A	B	C	Max	Min																										
10	25	90	45	90	25																										
20	5	5	5	5	5																										
30	100	20	99	100	20																										
40	3	8	1	8	1																										

• EDA Playground Link:

<https://edaplayground.com/x/989v>

10. Sorting Array in Ascending Order (Bubble Sort)

Design Code	Testbench Code
<pre> module sort_array; integer arr [0:7]; integer i, j, temp; initial begin arr[0] = 34; arr[1] = 7; arr[2] = 23; arr[3] = 32; arr[4] = 5; arr[5] = 62; arr[6] = 14; arr[7] = 19; \$display("Original Array:"); for (i = 0; i < 8; i = i + 1) \$write("%0d ", arr[i]); \$display(""); for (i = 0; i < 7; i = i + 1) for (j = 0; j < 7 - i; j = j + 1) if (arr[j] > arr[j+1]) begin temp = arr[j]; arr[j] = arr[j+1]; arr[j+1] = temp; end end \$display("Sorted Array:"); for (i = 0; i < 8; i = i + 1) \$write("%0d ", arr[i]); \$display(""); end endmodule </pre>	<pre> module sort_array_tb; sort_array uut(); initial begin #1; \$finish; end endmodule </pre>
Output <pre> Original Array: 34 7 23 32 5 62 14 19 Sorted Array: 5 7 14 19 23 32 34 62 </pre>	

- EDA Playground Link:
<https://edaplayground.com/x/BiZ3>

11. Left-aligned triangle star pattern

- **Code:**

```
module left_triangle;

    integer i, j;

    initial begin
        for (i = 1; i <= 4; i = i + 1) begin
            for (j = 1; j <= i; j = j + 1) begin
                $write("* ");
            end
            $write("\n");
        end

        $finish;
    end

endmodule
```

- **Output:**

```
*
* *
* * *
* * * *
```

- **EDA Playground Link:**

<https://edaplayground.com/x/gcAh>

12. Right-aligned triangle star pattern

- **Code:**

```
module right_aligned_triangle_pattern;

    integer i, j, space;

    initial
    begin
        for (i = 1; i <= 4; i = i + 1)
        begin
            // Print leading spaces
            for (space = 4 - i; space > 0; space = space - 1)
            begin
                $write(" "); // Two spaces for alignment
            end
            // Print stars
            for (j = 1; j <= i; j = j + 1)
            begin
                $write("* ");
            end
            $write("\n");
        end
        $finish;
    end

endmodule
```

- **Output:**

```
    *
   **
  ***
 ****
```

- **EDA Playground Link:**

<https://edaplayground.com/x/dvGm>

13. Inverted left triangle star pattern

- **Code:**

```
module inverted_left_triangle_pattern;

    integer i, j;

    initial
    begin
        for (i = 4; i >= 1; i = i - 1)
        begin
            for (j = 1; j <= i; j = j + 1)
            begin
                $write("* ");
            end
            $write("\n");
        end

        $finish;
    end
endmodule
```

- **Output:**

```
* * * *
* * *
* *
*
```

- **EDA Playground Link:**

<https://edaplayground.com/x/FEZZ>

14. Inverted right triangle star pattern

- **Code:**

```
module inverted_right_triangle_pattern;

    integer i, j, space;

    initial
    begin
        for (i = 0; i < 4; i = i + 1)
        begin
            // Print leading spaces
            for (space = 0; space < i; space = space + 1)
            begin
                $write(" "); // Two spaces for alignment
            end

            // Print stars
            for (j = 4; j > i; j = j - 1)
            begin
                $write("* ");
            end

            // Move to next line
            $write("\n");
        end

        $finish;
    end
endmodule
```

- **Output:**

```
* * * *
 * * *
  * *
   *
```

- **EDA Playground Link:**

<https://edaplayground.com/x/L6a3>

15. Pyramid star pattern

- **Code:**

```
module pyramid_star_pattern;
  integer i, j, space;
  parameter N = 4; // Number of rows

  initial
  begin
    for (i = 1; i <= N; i = i + 1)
    begin
      // Print leading spaces
      for (space = 1; space <= N - i; space = space + 1)
        $write(" "); // two spaces for alignment

      // Print stars with spaces
      for (j = 1; j <= i; j = j + 1)
        $write("* "); // star followed by two spaces

      $write("\n");
    end
  $finish;
end
endmodule
```

- **Output:**

```
  *
 * *
* * *
* * * *
```

- **EDA Playground Link:**

<https://edaplayground.com/x/KBFa>

16. Inverted pyramid star pattern

- **Code:**

```
module inverted_pyramid_pattern;
  integer i, j, space;
  parameter N = 4; // Number of rows

  initial
  begin
    for (i = 0; i < N; i = i + 1)
    begin
      // Print leading spaces
      for (space = 0; space < i; space = space + 1)
        $write(" ");

      // Print stars with spaces
      for (j = i; j < N; j = j + 1)
        $write("* ");

      $write("\n");
    end
    $finish;
  end
endmodule
```

- **Output:**

```
* * * *
* * *
* *
*
```

- **EDA Playground Link:**

<https://edaplayground.com/x/czxK>

17. Diamond star pattern

- Code:**

```
module diamond_pattern;

integer i, j, space;
integer N = 4; // Height of top half (total height = 2*N - 1)

initial
begin
    // Upper half of diamond (including middle row)
    for (i = 1; i <= N; i = i + 1) begin
        // Print spaces
        for (space = N - i; space > 0; space = space - 1)
            $write(" ");
        // Print stars
        for (j = 1; j <= i; j = j + 1)
            $write("* ");
        $write("\n");
    end

    // Lower half of diamond
    for (i = N - 1; i >= 1; i = i - 1)
    begin
        // Print spaces
        for (space = N - i; space > 0; space = space - 1)
            $write(" ");
        // Print stars
        for (j = 1; j <= i; j = j + 1)
            $write("* ");
        $write("\n");
    end

    $finish;
end

endmodule
```

- Output:**

```
*
* *
* * *
* * * *
* * *
* *
*
```

- EDA Playground Link:**

<https://edaplayground.com/x/WxEC>

18. Left aligned triangle pattern

- **Code:**

```
module pattern1;  
    integer i, j;  
  
    initial  
        begin  
            for (i = 1; i <= 4; i = i + 1)  
                begin  
                    for (j = 1; j <= i; j = j + 1)  
                        $write("%0d ", j);  
                    $write("\n");  
                end  
            $finish;  
        end  
endmodule
```

- **Output:**

```
1  
1 2  
1 2 3  
1 2 3 4
```

- **EDA Playground Link:**

<https://edaplayground.com/x/vewQ>

19. Right aligned triangle pattern

- **Code:**

```
module pattern2;
    integer i, j, space;

    initial
        begin
            for (i = 1; i <= 4; i = i + 1)
                begin
                    for (space = 4 - i; space > 0; space = space - 1)
                        $write(" "); // double space for alignment
                    for (j = i; j >= 1; j = j - 1)
                        $write("%0d ", j);
                    $write("\n");
                end
            $finish;
        end
    endmodule
```

- **Output:**

```
    1
   2 1
  3 2 1
 4 3 2 1
```

- **EDA Playground Link:**

<https://edaplayground.com/x/ZhCZ>

20. Inverted left aligned triangle pattern

- **Code:**

```
module pattern3;  
    integer i, j;  
  
    initial  
    begin  
        for (i = 4; i >= 1; i = i - 1)  
        begin  
            for (j = 1; j <= i; j = j + 1)  
                $write("%0d ", j);  
            $write("\n");  
        end  
        $finish;  
    end  
endmodule
```

- **Output:**

```
1 2 3 4  
1 2 3  
1 2  
1
```

- **EDA Playground Link:**

<https://edaplayground.com/x/n87S>

21. Inverted right aligned triangle pattern

- **Code:**

```
module pattern4;
    integer i, j, space;

    initial
        begin
            for (i = 4; i >= 1; i = i - 1)
                begin
                    for (space = 4 - i; space > 0; space = space - 1)
                        $write(" "); // double space for alignment
                    for (j = i; j >= 1; j = j - 1)
                        $write("%0d ", j);
                    $write("\n");
                end
            $finish;
        end
    endmodule
```

- **Output:**

```
4 3 2 1
 3 2 1
 2 1
 1
```

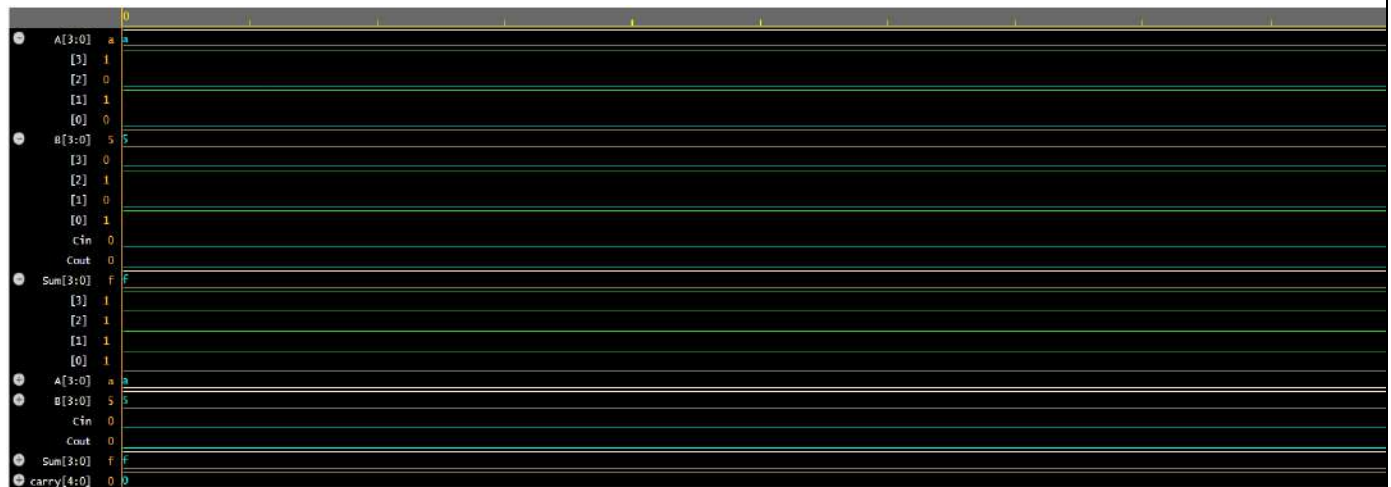
- **EDA Playground Link:**

<https://edaplayground.com/x/S9JA>

22. Adder using generate-for

Design Code	Testbench Code
<pre> module adder_generate_for #(parameter N = 4)(A, B, Cin, Sum, Cout); input [N-1:0] A, B; input Cin; output [N-1:0] Sum; output Cout; wire [N:0] carry; assign carry[0] = Cin; genvar i; generate for(i = 0; i < N; i = i + 1) begin : full_adder assign {carry[i+1], Sum[i]} = A[i] + B[i] + carry[i]; end endgenerate assign Cout = carry[N]; endmodule </pre>	<pre> module adder_generate_for_tb; reg [3:0] A, B; reg Cin; wire [3:0] Sum; wire Cout; adder_generate_for dut(A, B, Cin, Sum, Cout); initial begin A = 4'b1010; B = 4'b0101; Cin = 0; #10; \$display("A=%b B=%b Sum=%b Cout=%b", A, B, Sum, Cout); A = 4'b1111; B = 4'b1111; Cin = 1; #10; \$display("A=%b B=%b Sum=%b Cout=%b", A, B, Sum, Cout); end initial begin \$dumpfile("adder_generate_for.vcd"); \$dumpvars(0, adder_generate_for_tb); end endmodule </pre>
Output	
<pre> A=1010 B=0101 Sum=1111 Cout=0 A=1111 B=1111 Sum=1111 Cout=1 </pre>	

- **Waveform**



- **EDA Playground Link:**

<https://edaplayground.com/x/G8yP>

23. Adder using generate-if (Select between 2-bit and 4-bit Adder)

• Design Code	• Testbench Code
<pre> module adder_generate_if #(parameter USE_4BIT = 1)(input [3:0] A, B, input Cin, output [3:0] Sum, output Cout); generate if (USE_4BIT) begin : four_bit assign {Cout, Sum} = A + B + Cin; end else begin : two_bit assign {Cout, Sum[1:0]} = A[1:0] + B[1:0] + Cin; assign Sum[3:2] = 0; end endgenerate endmodule </pre>	<pre> module adder_generate_if_tb; reg [3:0] A, B; reg Cin; wire [3:0] Sum_4bit, Sum_2bit; wire Cout_4bit, Cout_2bit; // Instance 1: USE_4BIT = 1 adder_generate_if #(1) adder_4bit (.A(A), .B(B), .Cin(Cin), .Sum(Sum_4bit), .Cout(Cout_4bit)); // Instance 2: USE_4BIT = 0 adder_generate_if #(0) adder_2bit (.A(A), .B(B), .Cin(Cin), .Sum(Sum_2bit), .Cout(Cout_2bit)); initial begin \$display("Testing both 4-bit and 2-bit conditional generate adder"); A = 4'b0011; B = 4'b0101; Cin = 1; #10; \$display("Inputs: A=%b B=%b Cin=%b", A, B, Cin); \$display("4-bit -> Sum=%b Cout=%b", Sum_4bit, Cout_4bit); \$display("2-bit -> Sum=%b Cout=%b", Sum_2bit, Cout_2bit); A = 4'b1111; B = 4'b0001; Cin = 0; #10; \$display("\nInputs: A=%b B=%b Cin=%b", A, B, Cin); \$display("4-bit -> Sum=%b Cout=%b", Sum_4bit, Cout_4bit); \$display("2-bit -> Sum=%b Cout=%b", Sum_2bit, Cout_2bit); \$finish; end </pre>

```
initial
begin
    $dumpfile("adder_generate_if.vcd");
    $dumpvars(0, adder_generate_if_tb);
end
endmodule
```

- **Output**

Testing both 4-bit and 2-bit conditional generate adder

Inputs: A=0011 B=0101 Cin=1

4-bit -> Sum=1001 Cout=0

2-bit -> Sum=0001 Cout=1

Inputs: A=1111 B=0001 Cin=0

4-bit -> Sum=0000 Cout=1

2-bit -> Sum=0000 Cout=1

- **Waveform**



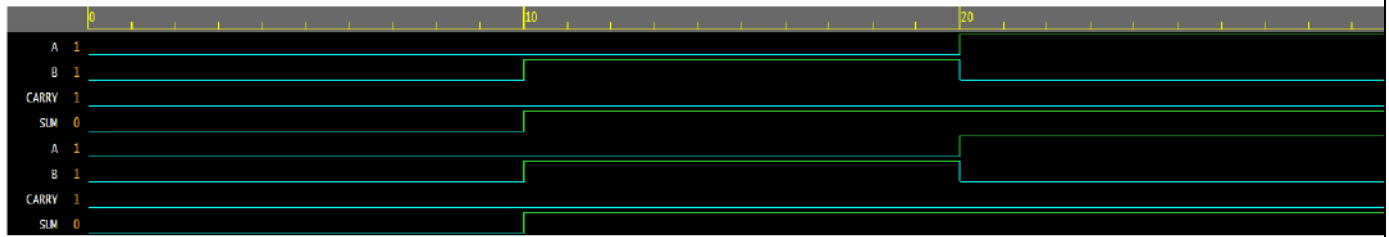
- **EDA Playground Link:**

<https://edaplayground.com/x/GF8G>

24. Half Adder using functions

Design Code	Testbench Code
<pre> module half_adder_func (A, B, SUM, CARRY); input wire A, B; output reg SUM, CARRY; // Function to compute SUM function automatic sum_func; input a, b; begin sum_func = a ^ b; end endfunction // Function to compute CARRY function automatic carry_func; input a, b; begin carry_func = a & b; end endfunction always @(A,B) begin SUM = sum_func(A, B); CARRY = carry_func(A, B); end endmodule </pre>	<pre> module half_adder_func_tb; reg A, B; wire SUM, CARRY; half_adder_func uut (A, B, SUM, CARRY); initial begin \$display("A B SUM CARRY"); \$monitor("%b %b %b %b", A, B, SUM, CARRY); A = 0; B = 0; #10 A = 0; B = 1; #10 A = 1; B = 0; #10 A = 1; B = 1; end initial begin \$dumpfile("half_adder_func.vcd"); \$dumpvars(0, half_adder_func_tb); end endmodule </pre>
Output	
<pre> A B SUM CARRY 0 0 0 0 0 1 1 0 1 0 1 0 1 1 0 1 </pre>	

- **Waveform**



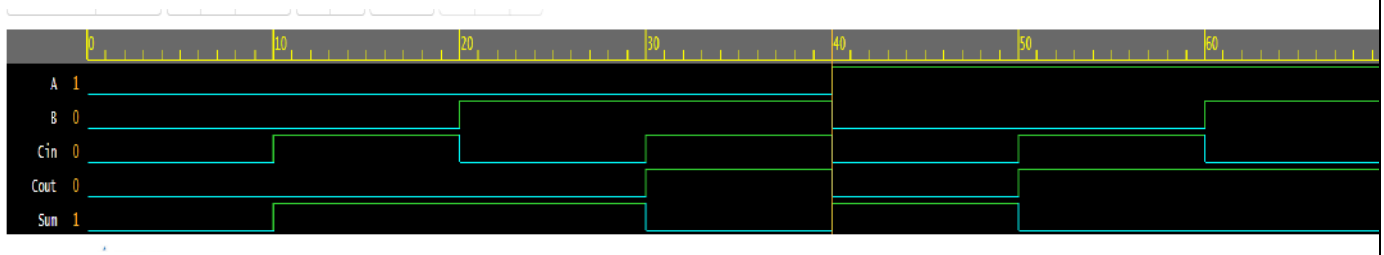
- **EDA Playground Link:**

<https://edaplayground.com/x/CKfR>

25. Full Adder using functions

• Design Code	• Testbench Code
<pre> module full_adder_using_function (A, B, Cin, Sum, Cout); input wire A, B, Cin; output wire Sum, Cout; // Function to compute sum and carry function [1:0] full_adder_func; input a, b, cin; begin full_adder_func[0] = a ^ b ^ cin; // Sum full_adder_func[1] = (a & b) (b & cin) (a & cin); // Carry end endfunction // Assign outputs assign {Cout, Sum} = full_adder_func(A, B, Cin); endmodule </pre>	<pre> module full_adder_using_function_tb; reg A, B, Cin; wire Sum, Cout; full_adder_using_function dut (A, B, Cin, Sum, Cout); initial begin \$display("A B Cin Sum Cout"); \$dumpfile("full_adder_wave.vcd"); \$dumpvars(0, full_adder_using_function_tb); for (integer i = 0; i < 8; i = i + 1) begin {A, B, Cin} = i[2:0]; #10; \$display("%b %b %b %b %b", A, B, Cin, Sum, Cout); end \$finish; end endmodule </pre>
• Output	
<pre> A B Cin Sum Cout 0 0 0 0 0 0 0 1 1 0 0 1 0 1 0 0 1 1 0 1 1 0 0 1 0 1 0 1 0 1 1 1 0 0 1 1 1 1 1 1 </pre>	

- **Waveform**



- **EDA Playground Link:**

<https://edaplayground.com/x/BF7e>

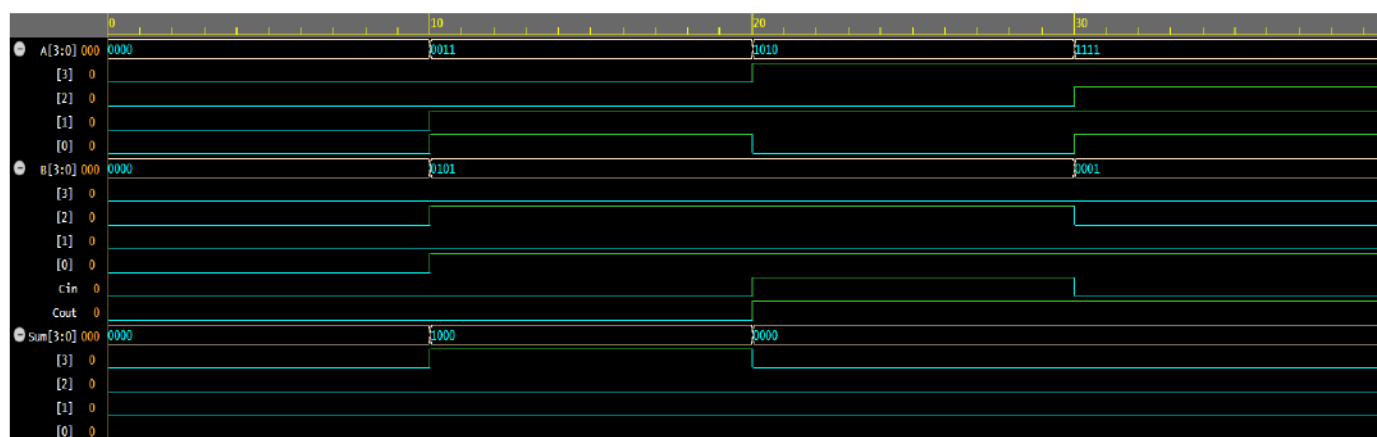
26. Ripple Carry Adder using function (4 bits)

• Design Code	• Testbench Code
<pre> module ripple_carry_adder_function (A, B, Cin, Sum, Cout); input [3:0] A, B; input Cin; output [3:0] Sum; output Cout; reg [4:0] carry; // Carry[0] = Cin, Carry[4] = Cout reg [3:0] sum_reg; assign Sum = sum_reg; assign Cout = carry[4]; // Full Adder Function function [1:0] full_adder_func; input a, b, cin; begin full_adder_func[0] = a ^ b ^ cin; // Sum full_adder_func[1] = (a & b) (b & cin) (a & cin); // Carry end endfunction integer i; always @(A, B, Cin) begin carry[0] = Cin; for (i = 0; i < 4; i = i + 1) begin {carry[i+1], sum_reg[i]} = full_adder_func(A[i], B[i], carry[i]); end end endmodule </pre>	<pre> module ripple_carry_adder_function_tb; reg [3:0] A, B; reg Cin; wire [3:0] Sum; wire Cout; ripple_carry_adder_function uut (A, B, Cin, Sum, Cout); initial begin \$display("A B Cin Sum Cout"); \$monitor("%b %b %b %b %b", A, B, Cin, Sum, Cout); \$dumpfile("ripple_adder_wave.vcd"); \$dumpvars(0, ripple_carry_adder_function_tb); A = 4'b0000; B = 4'b0000; Cin = 0; #10; A = 4'b0011; B = 4'b0101; Cin = 0; #10; A = 4'b1010; B = 4'b0101; Cin = 1; #10; A = 4'b1111; B = 4'b0001; Cin = 0; #10; A = 4'b1111; B = 4'b1111; Cin = 1; #10; \$finish; end endmodule </pre>

- **Output**

A	B	Cin	Sum	Cout
0000	0000	0	0000	0
0011	0101	0	1000	0
1010	0101	1	0000	1
1111	0001	0	0000	1
1111	1111	1	1111	1

- **Waveform**



- **EDA Playground Link:**

<https://edaplayground.com/x/HmrG>

27. Function to check whether the number is prime or not

• Design Code	• Testbench Code																						
<pre> module prime_checker(num, is_prime); input [3:0] num; output reg is_prime; function automatic bit check_prime; input [3:0] n; integer i; begin if (n <= 1) check_prime = 0; else begin check_prime = 1; for (i = 2; i < n; i = i + 1) begin if (n % i == 0) check_prime = 0; end end end endfunction always @(num) begin is_prime = check_prime(num); end endmodule </pre>	<pre> module prime_checker_tb; reg [3:0] num; wire is_prime; prime_checker uut (num, is_prime); integer i; initial begin \$display("Number\tIs_Prime"); for (i = 0; i <= 15; i = i + 1) begin num = i; #5; \$display("%d\t%b", num, is_prime); end \$finish; end endmodule </pre>																						
• Output																							
<table> <thead> <tr> <th>Number</th><th>Is_Prime</th></tr> </thead> <tbody> <tr><td>0</td><td>0</td></tr> <tr><td>1</td><td>0</td></tr> <tr><td>2</td><td>1</td></tr> <tr><td>3</td><td>1</td></tr> <tr><td>4</td><td>0</td></tr> <tr><td>5</td><td>1</td></tr> <tr><td>6</td><td>0</td></tr> <tr><td>7</td><td>1</td></tr> <tr><td>8</td><td>0</td></tr> <tr><td>9</td><td>0</td></tr> </tbody> </table>		Number	Is_Prime	0	0	1	0	2	1	3	1	4	0	5	1	6	0	7	1	8	0	9	0
Number	Is_Prime																						
0	0																						
1	0																						
2	1																						
3	1																						
4	0																						
5	1																						
6	0																						
7	1																						
8	0																						
9	0																						

10	0
11	1
12	0
13	1
14	0
15	0

- **EDA Playground Link:**

<https://edaplayground.com/x/b4kc>

28. Function to check whether the number is even or not

• Design Code	• Testbench Code																																		
<pre> module even_odd_checker(num, is_even); input [3:0] num; output reg is_even; // Function to check evenness function automatic bit check_even; input [3:0] n; begin check_even = (n % 2 == 0); end endfunction always @(num) begin is_even = check_even(num); end endmodule </pre>	<pre> module even_odd_checker_tb; reg [3:0] num; wire is_even; even_odd_checker uut (num, is_even); integer i; initial begin \$display("Number\tEven?"); for (i = 0; i <= 15; i = i + 1) begin num = i; #5; \$display("%d\t%b", num, is_even); end \$finish; end endmodule </pre>																																		
• Output																																			
<table> <thead> <tr> <th>Number</th><th>Even</th></tr> </thead> <tbody> <tr><td>0</td><td>1</td></tr> <tr><td>1</td><td>0</td></tr> <tr><td>2</td><td>1</td></tr> <tr><td>3</td><td>0</td></tr> <tr><td>4</td><td>1</td></tr> <tr><td>5</td><td>0</td></tr> <tr><td>6</td><td>1</td></tr> <tr><td>7</td><td>0</td></tr> <tr><td>8</td><td>1</td></tr> <tr><td>9</td><td>0</td></tr> <tr><td>10</td><td>1</td></tr> <tr><td>11</td><td>0</td></tr> <tr><td>12</td><td>1</td></tr> <tr><td>13</td><td>0</td></tr> <tr><td>14</td><td>1</td></tr> <tr><td>15</td><td>0</td></tr> </tbody> </table>		Number	Even	0	1	1	0	2	1	3	0	4	1	5	0	6	1	7	0	8	1	9	0	10	1	11	0	12	1	13	0	14	1	15	0
Number	Even																																		
0	1																																		
1	0																																		
2	1																																		
3	0																																		
4	1																																		
5	0																																		
6	1																																		
7	0																																		
8	1																																		
9	0																																		
10	1																																		
11	0																																		
12	1																																		
13	0																																		
14	1																																		
15	0																																		

• EDA Playground Link:

<https://edaplayground.com/x/utTM>

29. Fibonacci Series using function

- **Code:**

```
module fibonacci_func;

// Function to calculate nth Fibonacci number
function automatic [31:0] fib;
    input [4:0] n;
    integer i;
    reg [31:0] a, b, temp;
    begin
        a = 0;
        b = 1;
        if (n == 0)
            fib = a;
        else
            if (n == 1)
                fib = b;
            else
                begin
                    for (i = 2; i <= n; i = i + 1)
                        begin
                            temp = a + b;
                            a = b;
                            b = temp;
                        end
                    fib = b;
                end
            end
    end
endfunction
```



```
integer j;  
initial  
begin  
  $display("Fibonacci Series:");  
  for (j = 0; j < 10; j = j + 1)  
    begin  
      $display("fib(%0d) = %0d", j, fib(j));  
    end  
  $finish;  
end  
endmodule
```

- **Output:**

Fibonacci Series:

```
fib(0) = 0  
fib(1) = 1  
fib(2) = 1  
fib(3) = 2  
fib(4) = 3  
fib(5) = 5  
fib(6) = 8  
fib(7) = 13  
fib(8) = 21  
fib(9) = 34
```

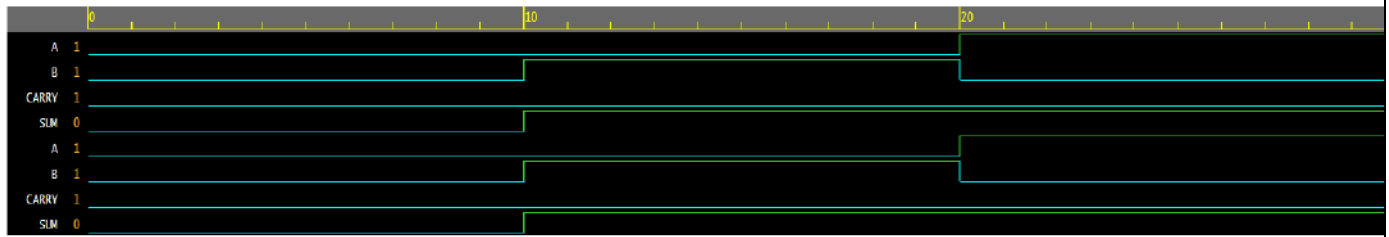
- **EDA Playground Link:**

<https://edaplayground.com/x/eJRw>

30. Half Adder using tasks

Design Code	Testbench Code
<pre> module half_adder_task (A, B, SUM, CARRY); input wire A, B; output reg SUM, CARRY; // Task to compute SUM and CARRY task automatic half_adder_task_logic; input a, b; output s, c; begin s = a ^ b; c = a & b; end endtask always @(A, B) begin half_adder_task_logic(A, B, SUM, CARRY); end endmodule </pre>	<pre> module half_adder_task_tb; reg A, B; wire SUM, CARRY; half_adder_task uut (A, B, SUM, CARRY); initial begin \$dumpfile("half_adder_task.vcd"); \$dumpvars(0, half_adder_task_tb); \$display("A B SUM CARRY"); \$monitor("%b %b %b %b", A, B, SUM, CARRY); A = 0; B = 0; #10; A = 0; B = 1; #10; A = 1; B = 0; #10; A = 1; B = 1; #10; \$finish; end endmodule </pre>
Output	
<pre> A B SUM CARRY 0 0 0 0 0 1 1 0 1 0 1 0 1 1 0 1 </pre>	

- **Waveform**



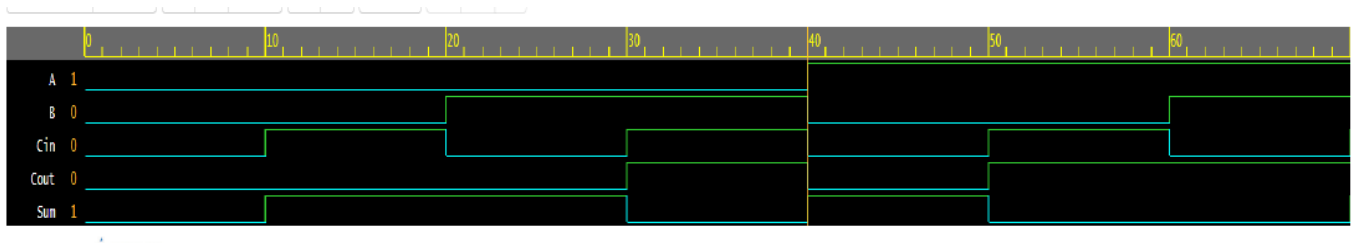
- **EDA Playground Link:**

<https://edaplayground.com/x/CyvE>

31. Full Adder using tasks

Design Code	Testbench Code
<pre> module full_adder_task (A, B, Cin, SUM, CARRY); input wire A, B, Cin; output reg SUM, CARRY; // Task to perform full adder logic task automatic full_adder_logic; input a, b, cin; output sum, carry; begin sum = a ^ b ^ cin; carry = (a & b) (b & cin) (a & cin); end endtask always @(A,B,Cin) begin full_adder_logic(A, B, Cin, SUM, CARRY); end endmodule </pre>	<pre> module full_adder_task_tb; reg A, B, Cin; wire SUM, CARRY; full_adder_task dut (A, B, Cin, SUM, CARRY); initial begin \$dumpfile("full_adder_task.vcd"); \$dumpvars(0, full_adder_task_tb); \$display("A B Cin SUM CARRY"); \$monitor("%b %b %b %b %b", A, B, Cin, SUM, CARRY); A=0; B=0; Cin=0; #10; A=0; B=0; Cin=1; #10; A=0; B=1; Cin=0; #10; A=0; B=1; Cin=1; #10; A=1; B=0; Cin=0; #10; A=1; B=0; Cin=1; #10; A=1; B=1; Cin=0; #10; A=1; B=1; Cin=1; #10; \$finish; end endmodule </pre>
Output	
<pre> A B Cin Sum Cout 0 0 0 0 0 0 0 1 1 0 0 1 0 1 0 0 1 1 0 1 1 0 0 1 0 1 0 1 0 1 1 1 0 0 1 1 1 1 1 1 </pre>	

- **Waveform**



- **EDA Playground Link:**

<https://edaplayground.com/x/EbFP>

32. Clock generation using task

- **Code:**

```
module clock_gen_task_tb;
    reg clk;

    // Task to generate clock signal
    task automatic generate_clock;
        ref clk_out;
        input integer cycles;
        integer i;
        begin
            for (i = 0; i < cycles; i = i + 1)
                begin
                    clk_out = 1;
                    #5;
                    clk_out = 0;
                    #5;
                end
            end
        endtask

    initial
        begin
            $dumpfile("clock_gen.vcd");
            $dumpvars(0, clock_gen_task_tb);

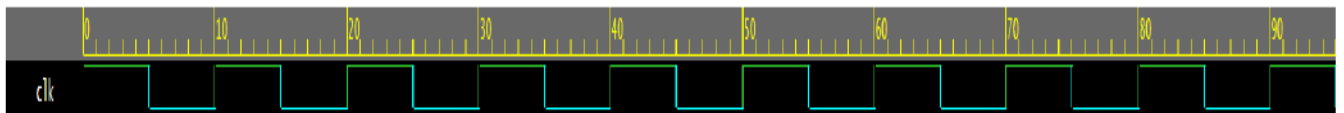
            clk = 0;

            // Call the clock generation task for 10 cycles
            generate_clock(clk, 10);

            #10; // Wait to observe last clock edge
```

```
$finish;  
end  
endmodule
```

- **Output:**



- **EDA Playground Link:**

https://edaplayground.com/x/w5B_

33. Multiple ways to generate a clock

- **Code:**

```
module all_clock_methods_tb;

parameter PERIOD = 10;
integer i;

reg clk1, clk2, clk3, clk4, clk5;
reg clk6, clk7, clk8, clk9;

// 1. Always block (most common)
initial clk1 = 0;
always #(PERIOD/2) clk1 = ~clk1;

// 2. Initial + forever
initial
begin
    clk2 = 0;
    forever #(PERIOD/2) clk2 = ~clk2;
end

// 3. Initial + repeat loop
initial
begin
    clk3 = 0;
    repeat (10)
    begin
        #(PERIOD/2) clk3 = ~clk3;
    end
end
```


// 4. Fork-join

initial

fork

begin

clk4 = 0;

forever #(PERIOD/2) clk4 = ~clk4;

end

join_none

// 5. Generate block

genvar g;

generate

initial

begin

clk5 = 0;

forever #(PERIOD/2) clk5 = ~clk5;

end

endgenerate

// 6. While loop

initial

begin

clk6 = 0;

i = 0;

while (i < 10)

begin

#(PERIOD/2) clk6 = ~clk6;

i = i + 1;

end

end

// 7. For loop

initial

```
begin
    clk7 = 0;
    for (i = 0; i < 10; i = i + 1)
        begin
            #(PERIOD/2) clk7 = ~clk7;
        end
    end

// 8. Function-based toggle
function automatic bit toggle(input bit state);
    toggle = ~state;
endfunction

initial
begin
    clk8 = 0;
    for (i = 0; i < 10; i = i + 1)
        begin
            #(PERIOD/2) clk8 = toggle(clk8);
        end
    end

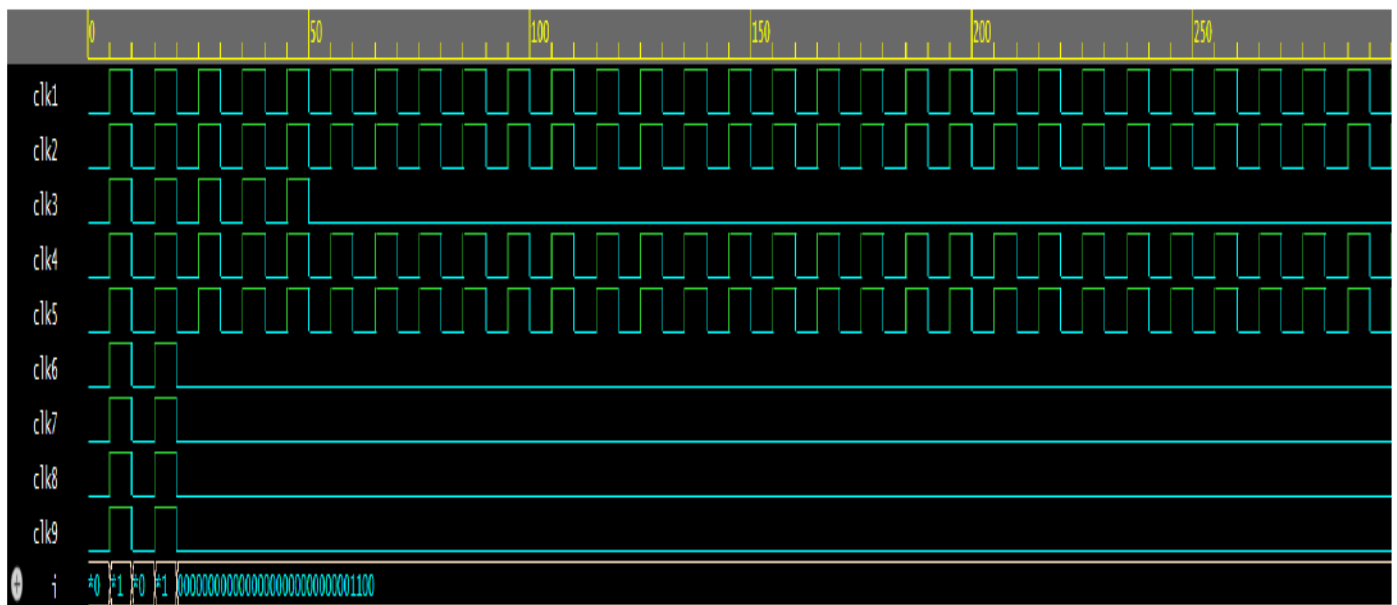
// 9. Manual toggle with delay
initial
begin
    clk9 = 0;
    #(PERIOD/2);
    clk9 = 1;
    #(PERIOD/2);
    clk9 = 0;
    #(PERIOD/2);
    clk9 = 1;
    #(PERIOD/2);
```

```
    clk9 = 0;
end

initial
begin
    $dumpfile("all_clocks.vcd");
    $dumpvars(0, all_clock_methods_tb);
    #300 $finish;
end

endmodule
```

- **Output:**



- **EDA Playground Link:**

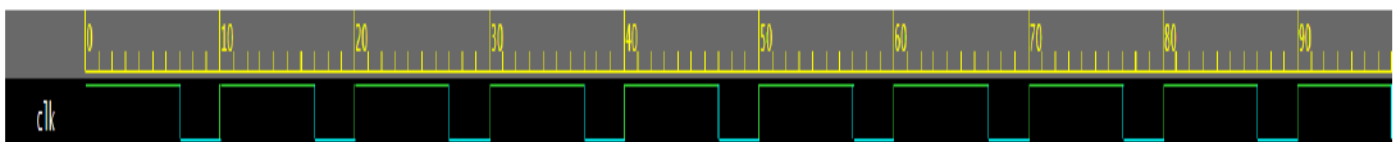
<https://edaplayground.com/x/qBgC>

34. Clock generation of 70% duty cycle

- **Code:**

```
module clock_70_duty;  
    reg clk;  
  
    initial clk = 0;  
  
    always  
    begin  
        clk = 1;    // Clock HIGH for 7 ns  
        #7;  
        clk = 0;    // Clock LOW for 3 ns  
        #3;  
    end  
  
    initial  
    begin  
        $dumpfile("clock70.vcd");  
        $dumpvars(0, clock_70_duty);  
        #100 $finish;  
    end  
endmodule
```

- **Output:**



- **EDA Playground Link:**

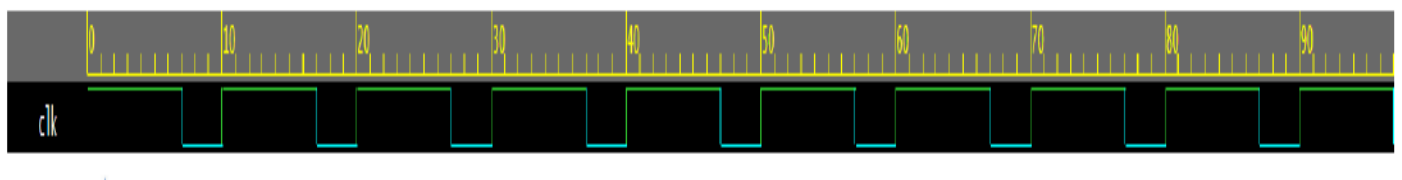
<https://edaplayground.com/x/ZH5P>

35. Clock generation of 40% duty cycle

- **Code:**

```
module clock_40_duty;  
    reg clk;  
  
    initial clk = 0;  
  
    always  
    begin  
        clk = 1; // HIGH for 4 ns  
        #4;  
        clk = 0; // LOW for 6 ns  
        #6;  
    end  
  
    initial  
    begin  
        $dumpfile("clock40.vcd");  
        $dumpvars(0, clock_40_duty);  
        #100 $finish;  
    end  
endmodule
```

- **Output:**



- **EDA Playground Link:**

<https://edaplayground.com/x/ajHz>

36. Clock generation for 5ns time period

- **Code:**

```
`timescale 1ns/1ps

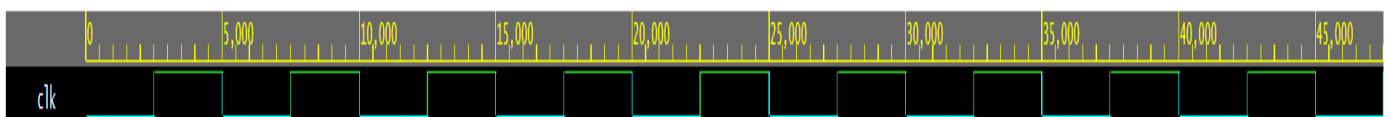
module clk_5ns_period;
    reg clk;

    initial clk = 0;

    always
    begin
        #2.5 clk = ~clk;
    end

    initial begin
        $dumpfile("clk_5ns.vcd");
        $dumpvars(0, clk_5ns_period);
        #50 $finish;
    end
endmodule
```

- **Output:**



- **EDA Playground Link:**

<https://edaplayground.com/x/hDdc>

37. Clock generation for 6.6ns time period

- **Code:**

```
`timescale 1ns/1ps
module clk_6_6ns_period;

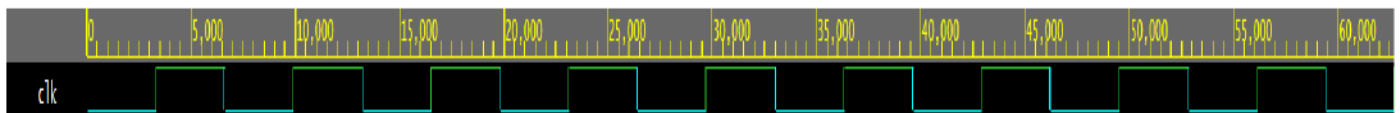
  reg clk;

  initial clk = 0;

  always
  begin
    #3.3 clk = ~clk;
  end

  initial
  begin
    $dumpfile("clk_6_6ns.vcd");
    $dumpvars(0, clk_6_6ns_period);
    #66 $finish;
  end
endmodule
```

- **Output:**



- **EDA Playground Link:**

<https://edaplayground.com/x/uYzi>

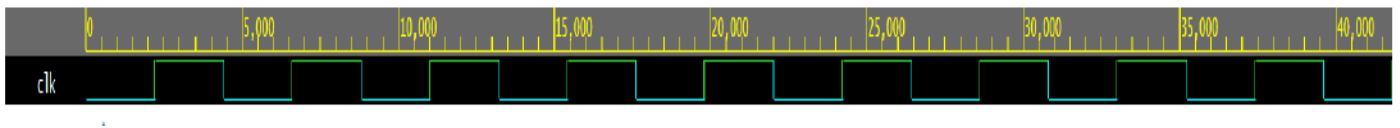
38. Clock generation for 4.4microsecond time period

- **Code:**

```
`timescale 1us/1ns
```

```
module clk_4_4us_period;  
    reg clk = 0;  
    always #2.2 clk = ~clk;  
  
    initial  
    begin  
        $dumpfile("clk_4_4us.vcd");  
        $dumpvars(0, clk_4_4us_period);  
        #44 $finish;  
    end  
endmodule
```

- **Output:**



- **EDA Playground Link:**

<https://edaplayground.com/x/ZH6S>

39. Verilog code to generate 10 unique random numbers between 0-500

- **Code:**

```
module unique_random_numbers;

integer rand_nums[0:9]; // Array to store 10 unique numbers
integer i, j;
integer temp;
reg is_unique;

initial
begin
    $display("10 unique random numbers");

    i = 0;
    while (i < 10)
    begin
        temp = $random % 501; // range: 0–500
        if (temp < 0)
            temp = -temp; // Make it positive
        is_unique = 1;

        // Check for duplicates
        for (j = 0; j < i; j = j + 1)
        begin
            if (rand_nums[j] == temp)
            begin
                is_unique = 0;
                break;
            end
        end
    end
end
```

```
    if (is_unique)
    begin
        rand_nums[i] = temp;
        $display("Number %0d: %0d", i + 1, temp);
        i = i + 1;
    end
end

$finish;
end
endmodule
```

- **Output:**

10 unique random numbers

Number 1: 248

Number 2: 199

Number 3: 239

Number 4: 309

Number 5: 457

Number 6: 471

Number 7: 214

Number 8: 361

Number 9: 29

Number 10: 318

- **EDA Playground Link:**

<https://edaplayground.com/x/6htM>

40. Verilog code to generate 20 unique random numbers between 0-1000 in ascending order

- **Code:**

```
module random_unique_sorted;

integer number[0:19]; // Array to hold 20 unique numbers
integer i, j, temp, num;
integer is_unique;

initial
begin
i = 0;
while (i < 20)
begin
num = {$random} % 1001; // range 0 to 1000

// Check for uniqueness
is_unique = 1;
for (j = 0; j < i; j = j + 1)
begin
if (number[j] == num)
begin
is_unique = 0;
break;
end
end

// If unique, store the number
if (is_unique == 1)
begin
number[i] = num;
i = i + 1;
end
end
end
```

```
end

// Bubble sort to sort the array
for (i = 0; i < 20; i = i + 1)
begin
    for (j = i + 1; j < 20; j = j + 1)
        begin
            if (number[i] > number[j])
                begin
                    temp = number[i];
                    number[i] = number[j];
                    number[j] = temp;
                end
            end
        end
    end
end

$display("20 Unique Random Numbers (0 to 1000) in Ascending Order:");
for (i = 0; i < 20; i = i + 1)
begin
    $display("Number[%0d] = %0d", i, number[i]);
end
end
endmodule
```

- **Output:**

20 Unique Random Numbers (0 to 1000) in Ascending Order:

```
Number[0] = 86
Number[1] = 96
Number[2] = 98
Number[3] = 129
Number[4] = 229
Number[5] = 251
Number[6] = 272
Number[7] = 278
Number[8] = 338
```

Number[9] = 369
Number[10] = 371
Number[11] = 394
Number[12] = 500
Number[13] = 652
Number[14] = 672
Number[15] = 673
Number[16] = 693
Number[17] = 848
Number[18] = 981
Number[19] = 994

- **EDA Playground Link:**

<https://edaplayground.com/x/KfZc>